ЦСО · МАТЕРИАЛ ДЛЯ САМОСТОЯТЕЛЬНОГО? ИЗУЧЕНИЯ

# Agent Coding

## Подход, инструменты и процесс

Как с помощью AI-агента превратить идею бизнеса в готовый код — и как устроить репозиторий, чтобы агент работал надёжно.

Все примеры вымышленные. Показываем подход, а не конкретные сервисы.

Собрано тем же процессом, который описывает

# Для инженеров, которым нужна предсказуемость

| | | |
|---|---|---|
| 01 | **Для кого** | Инженеры, которые встраивают AI-агента в ежедневную работу — чтобы получать предсказуемый результат, а не «поиграть». |
| 02 | **Как читать** | По порядку: проблема и идея конвейера → фазы → архитектура. Схемы — опора, текст — пояснение. |
| 03 | **Что забрать** | Модель из трёх уровней, шпаргалку «фаза → скилл → артефакт» и минимальный план старта. |

# Наивный «промпт → код» ломается трижды

Один длинный промпт ломается предсказуемо. Каждую поломку закрывает своя часть конвейера.

01

## Дрейф терминов и допущений

Ты говоришь «клиент», агент пишет account. Придумывает поля и связи, которых нет в модели.

Лечим **контекстом**

02

## Код не там и не той формы

Логика висит на эндпоинте. Бизнес-правила размазаны по слоям. Диф нечитаем — его нельзя ревьюить.

Лечим **архитектурой**

03

## Нет проверки

Агент пишет «Готово», а задача не сделана: тесты не гоняли, ревью не было.

Лечим **процессом**

# Конвейер фаз, а не один длинный промпт

## На выходе каждой фазы — готовый артефакт. Он же вход следующей.

| | |
|---|---|
| **Вход → выход** | каждая фаза работает по одной схеме |
| **Свой класс провалов** | каждая фаза закрывает конкретный тип ошибок |
| **Дёшево повторить** | перезапустить фазу дёшево, переписать всё — дорого |

ВТОРАЯ ИДЕЯ

### Архитектура — интерфейс агента

Архитектура и документы репозитория — это то, через что агент понимает систему. Без них конвейер упирается в неоднозначность.

# Идея проходит три уровня — от бизнеса до кода

УРОВЕНЬ 1

## Идея

Бизнес пишет задачу-эпик в трекере.

→ Эпик в трекере



УРОВЕНЬ 2

## Планирование

Команда раскладывает эпик в структуре и языке наших сервисов.

→ Единый spec + тикеты



УРОВЕНЬ 3

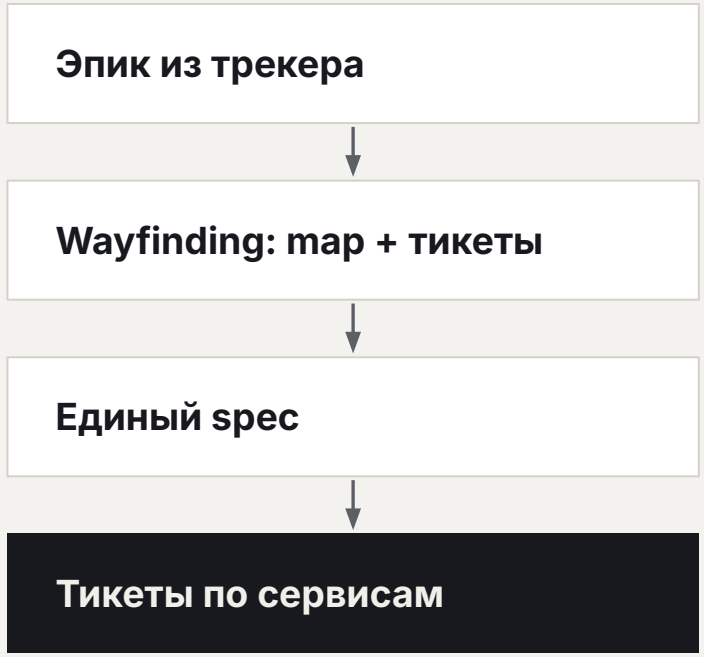
## Реализация

Команда детально прорабатывает задачу внутри каждого сервиса.

→ Код, тесты, ревью

**Чем больше номер уровня, тем ближе к коду.** Один spec уровня 2 запускает N прогонов уровня 3 — по одному на каждый затронутый сервис.

# Док-хаб и wayfinding: планируем картой




Док-хаб — репозиторий с описаниями и глоссариями сервисов. Кода нет. CONTEXT-MAP.md ведёт к CONTEXT.md каждого сервиса.

ШАГИ

1. Заводим карту .scratch/<effort>/map.md
2. Режем идею на исследовательские тикеты
3. Резолвим тикеты по одному
4. Копим «Decisions so far», туман превращаем в явные вопросы

«Мы не знаем X» превращается в тикет, а не в сюрприз посреди разработки.

```
# Map: Интеграция Поручений

## Destination
Единый spec + тикеты по сервисам

## Decisions so far
- T01: сводка схем по 7 репо
- T02: RMQ-события, row_hash

## Not yet specified (fog)
- Формат хендшейка dzo_appeal_id

## Out of scope
- Реализация кода
```

# Тикет L2 — изменение контракта, а не кода

### Внешние рёбра

API и события — то, что видят соседи. Отсюда Blocked by между сервисами.

### Внутренние рёбра

Изменения схемы БД, которые гейтят следующий тикет того же сервиса. Помечаем, чтобы не приняли за контракт.

### Контракт определяет потребитель

Пишем не «что сервис делает», а «что нужно соседу».

### Совместимость — решение L2

Аддитивное или ломающее изменение влияет на соседей, поэтому решаем здесь.

**Готово,** когда сервис может запустить свой конвейер L3.

```
# <Сервис> — <сценарий>
Status: ready-for-agent
Blocked by: <тикеты>

## Граница
API: <методы + формат>
События: <топик + payload>
## Поведение (end-to-end)
## Внутренние опоры  ← если есть
## Acceptance  ← на границе
## Ссылки  ← Spec · Эпик L1
```

# Проектирование — одна сессия до кода

`grill-with-docs` → `to-spec` → `to-tickets`

## Интервью

Агент задаёт по одному вопросу с рекомендацией, ищет факты в коде, решения оставляет тебе. Термины уточняет, конфликты с глоссарием вскрывает сразу.

→ `CONTEXT.md, docs/adr/*`

## Синтез спеки

Собирает спеку из уже сказанного, без нового интервью. Берёт термины из глоссария, соблюдает ADR, выбирает швы для тестов.

→ `PRD.md`

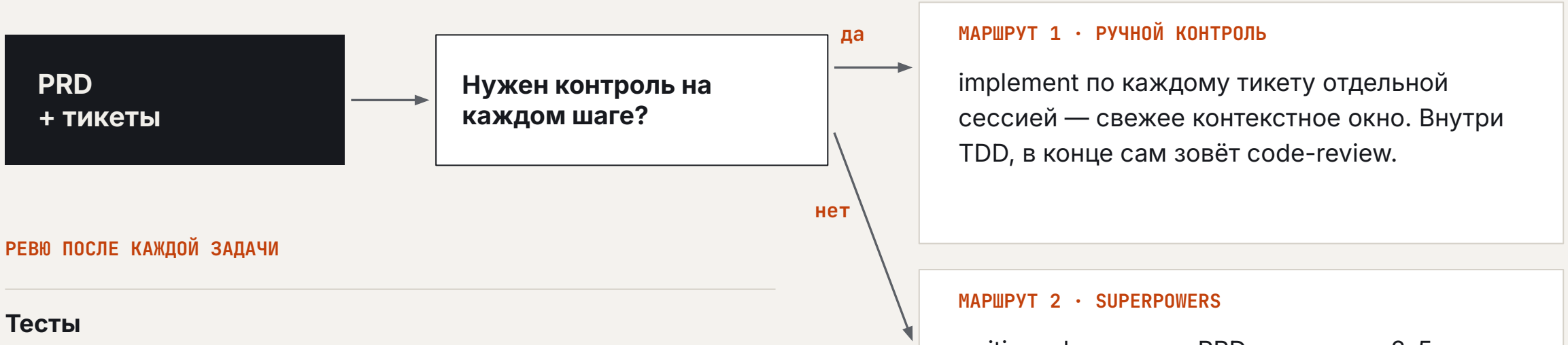
## Вертикальные срезы

Только для маршрута implement. Узкий полный путь через схему, API и тесты, размером в одно контекстное окно.

→ `issues/NN-*.md`

Артефакт прошлой фазы — вход следующей. Изменился PRD — заново собрали тикеты, не переписывая всё.

# Два маршрута — выбор по степени контроля




МАРШРУТ 1 · РУЧНОЙ КОНТРОЛЬ

implement по каждому тикету отдельной сессией — свежее контекстное окно. Внутри TDD, в конце сам зовёт code-review.

МАРШРУТ 2 · SUPERPOWERS

writing-plans режет PRD на шаги по 2–5 минут. subagent-driven-development: свежий субагент и ревью на каждую задачу, в конце — ревью ветки.

РЕВЮ ПОСЛЕ КАЖДОЙ ЗАДАЧИ

**Тесты**

не «написали», а прогнали

**`verification-before-completion`**

сначала вывод команд, потом «успех»

**`code-review`**

по рубрике репозитория и спеке

# Кодовая база — интерфейс агента

| НАИВНЫЙ РЕПОЗИТОРИЙ | РЕПОЗИТОРИЙ КАК ИНТЕРФЕЙС |
|---|---|
| О слоях договорились на словах. Код лежит где попало. | **У каждого типа кода ровно одно место и одна форма.** |
| Три слова для одного понятия. | **Термины зафиксированы в глоссарии.** |
| Решения помнят только люди. | **Решения записаны в ADR вместе с «когда ломать».** |
| Агент угадывает — и каждый раз по-разному. | **Агент находит место, а не изобретает его.** |

## ЧТО БЕРЁМ ИЗ DDD — АГЕНТ ДЕРЖИТ ТОЛЬКО КОНТЕКСТНОЕ ОКНО

### Ubiquitous language

один словарь на сервис

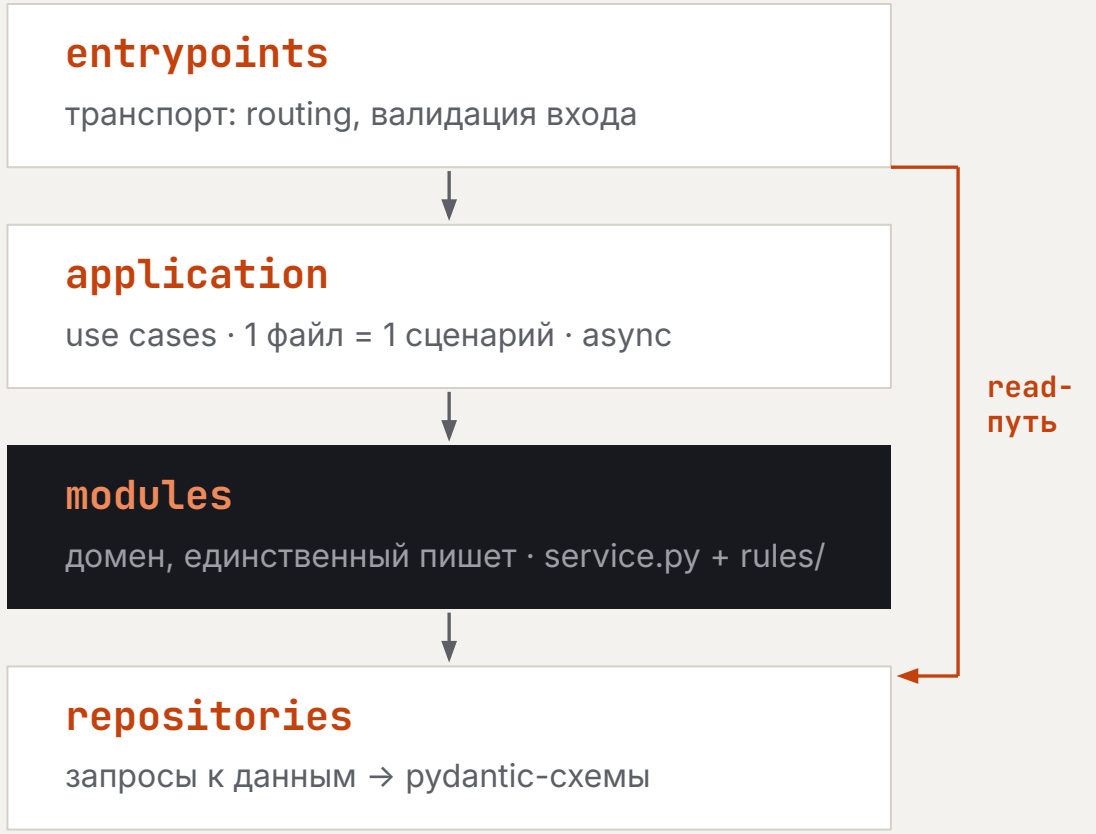
### Bounded context

один сервис — один глоссарий

### Domain isolation

домен не зависит от транспорта и БД

# У каждого типа кода ровно одно место




**ПРАВИЛА**

application только ведёт сценарий. Новый сценарий — новый файл, а не ветка в старом.

modules — единственный, кто пишет. service.py пишет в базу, но не ведёт сценарий.

rules/ — чистые sync-функции без базы и сети: is_*, validate_*, resolve_*.

Чтение идёт мимо modules: entrypoints → application → repositories.

Домен не бросает ошибки транспорта (HTTPException).

«Где это место?» — уже не творческий вопрос, а поиск по таблице.

# Каждое правило живёт ровно в одном документе

`CONTEXT.md`

## Глоссарий

Только язык, без реализации. Один канонический термин и список _Avoid_.

читает **весь цикл**

`docs/adr/`

## Решения

Только когда решение трудно откатить, оно удивляет без контекста и были реальные альтернативы.

читает **проектирование**

`architecture.md`

## Как писать код

Таблица «куда что класть и как назвать» + секция «когда ломать правило».

читает **до написания кода**

`CODING_STANDARDS.md`

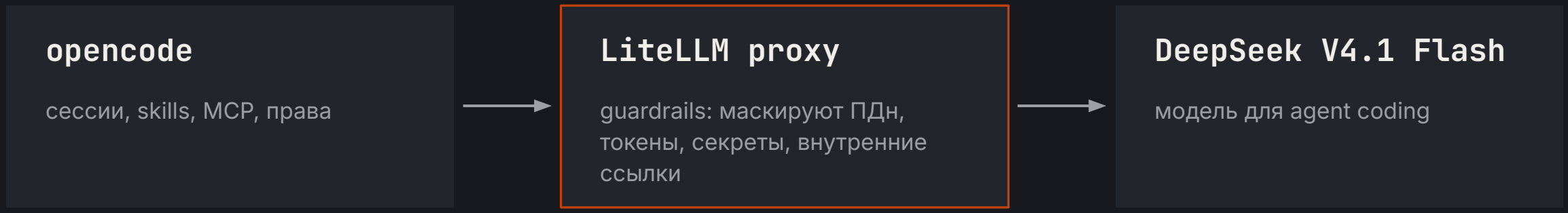
## Чек-лист ревью

Тот же свод правил для ревью. Ссылается на architecture.md, а не повторяет.

читает **при ревью**

Документы делятся по моменту в цикле агента: на каждой фазе он читает только нужное — меньше контекста.

# Одна точка входа и корпоративный шлюз




Напрямую к провайдеру агент не обращается — чувствительные данные остаются внутри контура.

---

### codegraph

Граф символов и вызовов. Агент находит нужный код без grep-циклов.

### Markdown-трекер

.scratch/ живёт в git: историю правок видно, её можно ревьюить.

### Скиллы

Matt Pocock skills и superpowers. Скилл — фаза конвейера, записанная как инструкция.

# Фаза → скилл → артефакт

| УР. | ФАЗА | СКИЛЛ | АРТЕФАКТ |
|---|---|---|---|
| 1 | **Эпик в трекере** | — | эпик в трекере задач |
| 2 | **Wayfinding** | wayfinder | map.md + исследовательские тикеты |
| | **Декомпозиция** | to-tickets | тикеты «сервис × сценарий», Blocked by |
| 3 | **Проектирование** | grill-with-docs | CONTEXT.md, docs/adr/* |
| | **Спека** | to-spec | PRD.md |
| | **Тикеты** | to-tickets | .scratch/<feature>/issues/NN-*.md |
| | **Исполнение · 1** | implement | код |
| | **Исполнение · 2** | writing-plans + SDD | план + код |
| | **Ревью** | code-review, verification | зелёные тесты, ревью |

# Сначала процесс, потом конвейер

1. Введи глоссарий CONTEXT.md — самый дорогой по эффекту шаг
2. Раздели доки по моменту: гид и чек-лист ревью
3. Напиши architecture.md: «куда что кладётся» + «когда ломать»
4. Один реальный прогон: grill → spec → tickets → implement → verify
5. Только потом подключай субагентный конвейер

Сопротивление — на шагах 1–3: это рутина, которая «не пишет код». Но именно она делает 4–5 предсказуемыми.

ИТОГ

**Конвейер, а не промпт**

**Три уровня: от идеи до кода**

**Архитектура — контракт**

**Один источник на правило**

**Ревью после каждой задачи**

# Инструменты и скиллы из материала

| | | |
|---|---|---|
| **opencode** | Среда, в которой идёт вся работа: сессии, skills, MCP, права | opencode.ai |
| **codegraph** | Граф символов и вызовов вместо grep-циклов | github.com/colbymchenry/codegraph |
| **LiteLLM** | Основа корпоративного proxy и шлюза к моделям; guardrails | github.com/BerriAI/litellm |
| **Matt Pocock skills** | grill-with-docs, to-spec, to-tickets, implement, wayfinder, code-review, triage | github.com/mattpocock/skills |
| **superpowers** | brainstorming, writing-plans, subagent-driven-development,verification-before-completion | github.com/obra/superpowers |